

Capability-Bounded Agents: Provable, Injection-Resistant Action Limits for Autonomous LLM Agents

Narendar Kumar Ale

PhD Student, Artificial Intelligence, University of the Cumberlands, KY, USA

`nale02820@cumberlands.edu`

July 2026

Abstract

Prompt injection is the highest-ranked unresolved risk for deployed LLM agents (OWASP LLM Top 10), and a recent formal result argues it may be unfixable from inside the model: any Contextual-Integrity-based in-model defense either misses attacks that construct a legitimate-looking context, or blocks genuinely legitimate behavior [1]. We argue that this impossibility result, rather than being discouraging, motivates a structural shift: safety for *bounded, irreversible actions* should not depend on the model resisting injection at all. We formalize a **capability contract** and an out-of-model **reference monitor** that mediates every tool call before execution, and prove that for any bounded action a , a executes $\implies a$ is permitted by the contract — a containment guarantee that holds regardless of what the model emits, adversarial or not. We validate the mechanism with (1) a containment benchmark against a modeled adversary (0% bypass vs. up to 100% for a prompt-only guardrail), and (2) a real-model adversarial evaluation against two production Claude models on Amazon Bedrock, in which every emitted tool call was independently checked against the monitor. We report an honest null result at current injection strength — both models resisted 100% of our direct injections — and situate it against external evidence that real-world indirect injection does succeed at non-trivial rates on other models [2], with real consequences when it does [8]. We conclude that in-model alignment and out-of-model enforcement are complementary, not substitutable, and that for bounded actions the latter is the only mechanism that offers a provable guarantee.

1 Introduction

Autonomous LLM agents increasingly act in the world: they call tools, execute code, send messages, and modify state, often with multiple steps between an initial instruction and a consequential action. This capability is also the attack surface. Prompt injection — text in a tool’s output, a retrieved document, or a user message that causes an agent to deviate from its intended task — is ranked the #1 risk in the OWASP Top 10 for LLM Applications, and 2026 has produced concrete evidence that it is not a theoretical concern: Microsoft disclosed two vulnerabilities in its Semantic Kernel agent framework (CVE-2026-25592, CVE-2026-26030) in which prompt injection escalated to host-level remote code execution [8].

The prevailing defense paradigm is *in-model*: instruction/data separation, refusal tuning, and output filtering, all of which rely on the model itself recognizing and resisting an injected instruction. Abdelnabi and Bagdasarian [1] recently gave a formal argument, via a Contextual Integrity (CI) framing, that this paradigm has a structural ceiling: an adversary can always construct a context in which a blocked information flow appears contextually legitimate, or a defender who tightens norms

enough to block it will also block genuinely legitimate agent behavior. In other words, in-model robustness cannot be made complete without also breaking the agent’s usefulness.

This paper takes that impossibility result as a premise rather than a conclusion to argue against. If safety cannot be proven to hold *inside* the model, then for the subset of actions whose consequences are unacceptable regardless of why the model attempted them (deleting a production resource, exfiltrating a secret), safety must be enforced *outside* the model, by a mechanism whose correctness does not depend on model behavior at all.

Contributions.

1. A formal **capability contract** and **reference monitor** construction (§4) with a containment theorem: for any bounded action, execution implies contract-permission, for all agent policies including adversarial ones.
2. A **mechanism benchmark** (§5) demonstrating the containment gap between prompt-guardrail and reference-monitor enforcement under a modeled adversary: 100%/50% bypass for prompt guardrails vs. 0% for the monitor.
3. A **real-model adversarial evaluation** (§6) against production Bedrock Claude models, with full raw transcripts published, reporting an honest result: 0% prompt-mode bypass observed at this injection strength — and a methodologically explicit discussion of what that result does and does not prove.
4. A synthesis (§7) connecting that result to external, independently documented real-world bypass evidence, resolving the apparent tension between “we found no bypass” and “bypasses are known to occur.”

Both artifacts are public: the core mechanism and mechanism benchmark at <https://github.com/NarenderKumarA/capability-bounded-agents>, and the real-model evaluation with raw transcripts at <https://github.com/NarenderKumarA/capability-bounded-agents-eval>.

2 Related Work

In-model defenses and their limits. Abdelnabi and Bagdasarian [1] is the most directly relevant prior work: their CI-based impossibility argument is the premise this paper builds on. Adversarial-suffix attacks [6] independently demonstrate that alignment/refusal training does not provide a hard robustness guarantee even against optimization-based attacks, reinforcing that in-model defenses are best modeled as empirically strong but not provably complete.

Empirical prevalence and realism of injection. Khodayari et al. [2] conducted a large-scale empirical study of indirect prompt injection across 1.2 billion crawled URLs, finding over 15,000 validated injection instances and measuring model compliance directly: across 5,200 trials over 13 models, compliance was “limited but non-negligible, reaching up to 8% for smaller models” on plain-text inputs. This is, to our knowledge, the most direct existing measurement of real-world prompt-mode bypass rate, and it is the number our own real-model evaluation (§6) should be read against. Choi et al. [3] identify a related but distinct vector, *agent data injection*, in which malicious *data* rather than instructions is disguised as trusted context (e.g. metadata), and report that “current agents do not isolate trusted data from untrusted data” across several production systems — directly motivating the untrusted-tool-output threat model we adopt in §3. Boisvert et

al. [4] introduce DoomArena, an evaluation framework for agent security threats, and report that guardrail-based defenses underperform relative to defenses that route through a separate, more capable evaluator — consistent with our motivation for moving the decision outside the acting model entirely.

Policy layers and zero-trust framings. Kholkar and Ahuja’s Policy-as-Prompt [5] is the closest prior art to this paper’s mechanism: it compiles governance documents into a policy tree that becomes a runtime classifier enforcing least privilege with an audit trail. It differs from our construction in kind, not just detail: their enforcement layer is itself a learned/prompted classifier, so its correctness is empirical, not provable, in the same sense as the reference monitor of §4 — the reference monitor concept itself is much older, originating in classical computer security as an access-mediation component whose correctness is established independently of the subject it mediates [7]. This paper’s contribution is to bring that classical, provable construction to LLM agent tool-calling specifically, and to pair it with a benchmark and a real-model evaluation rather than an architecture sketch alone.

3 Threat Model

We consider an agent with access to a fixed set of tools, driven by an LLM policy π that may be adversarially influenced through any channel that reaches the model’s context: a user message, the output of a previously invoked tool, or a retrieved document (RAG). We do *not* assume π is trusted, nor do we assume any particular robustness property of π — the containment guarantee in §4 is proven to hold for *all* possible π , including one fully compromised by injection. What is trusted is the **capability contract** itself: a static, human-authored specification of which tool calls are permitted, evaluated by a monitor outside the model’s control. We call an action **bounded** if it is destructive, irreversible, or crosses a trust boundary (e.g. deleting a resource, or sending data to an external recipient) — the class of actions for which “the model changed its mind after being tricked” is not an acceptable failure mode.

4 Formal Model

Definition 1 (Capability). *A capability is a tuple $(n, \text{allowed}, \phi)$ where n is a tool name, $\text{allowed} \in \{\top, \perp\}$, and ϕ is an optional predicate over call arguments. Absent an explicit capability for n , $\text{allowed} = \perp$ (default-deny).*

Definition 2 (Capability Contract). *A capability contract C is a finite set of capabilities, one per tool name. C is specified independently of, and prior to, any agent execution.*

Definition 3 (Reference Monitor). *A reference monitor M_C is a function $M_C(n, a) \rightarrow \{\text{permit}, \text{deny}\}$ such that $M_C(n, a) = \text{permit}$ iff C contains a capability for n with $\text{allowed} = \top$ and (ϕ is absent or $\phi(a) = \top$).*

We say an execution environment is **monitor-enforced** if a tool call (n, a) is passed to the underlying tool implementation only if $M_C(n, a) = \text{permit}$; otherwise it is **prompt-enforced** if the call is passed to the tool whenever the policy π emits it, with no independent check.

Theorem 1 (Containment). *For any policy π (including adversarially controlled ones) and any tool call (n, a) emitted by π in a monitor-enforced environment: if (n, a) executes, then $M_C(n, a) = \text{permit}$.*

Proof. By construction of the monitor-enforced executor: the underlying tool implementation for n is invoked only along the code path guarded by $M_C(n, a) = \text{permit}$; there is no other path to invocation. This holds independently of how (n, a) was produced — π ’s prompt, weights, or adversarial context are not inputs to M_C . ■ □

Corollary 1 (Contrapositive containment). *If $M_C(n, a) = \text{deny}$, then (n, a) never executes, for any π , regardless of context.*

Theorem 1 is a soundness guarantee, not a completeness one: it guarantees the monitor never permits what the contract forbids, but says nothing about whether the contract itself is correctly specified for the deployment’s actual security requirements. A monitor enforcing an overly permissive contract is sound but not safe in any useful sense — contract correctness is a specification problem outside the scope of this theorem, and we return to it as a limitation in §8.

5 Mechanism Benchmark: Modeled Adversary

As a first validation of the construction, we implement a small benchmark (`cba/harness.py`) with three tools (`read_status`, `delete_resource`, `send_message`) and four attacks spanning three injection vectors (poisoned tool output, poisoned retrieved document, direct user-turn social engineering), targeting either an unauthorized delete or an exfiltration of a planted secret. A capability contract permits only `read_status` and `send_message` to internal recipients. Because this benchmark’s purpose is to validate the *monitor*, not measure real model behavior, we model whether an injection “flips” a given policy via an intensity threshold rather than querying a real model — the real-model question is addressed separately in §6.

Policy	Enforcement	Bypass rate
vulnerable-7b	prompt guardrail	100%
vulnerable-7b	reference monitor	0%
hardened-frontier	prompt guardrail	50%
hardened-frontier	reference monitor	0%

Table 1: Mechanism benchmark: modeled adversary. Whenever an injection flips the modeled policy, the reference monitor still blocks the bounded action, independent of policy strength.

This confirms the mechanism does what Theorem 1 claims in a controlled setting, and motivates the natural next question: does the same hold with a real model in the loop, and does prompt-mode bypass actually occur?

6 Real-Model Adversarial Evaluation

6.1 Design

We replace the modeled flip-decision with a real one. Each of the four attacks from §5 is realized as an actual multi-turn conversation sent to a real model via the Bedrock InvokeModel API, using Claude’s native `tools/tool_use` mechanism rather than a prompt-convention or regex parse, so that “did the model attempt the forbidden call” is answered by inspecting structured output, not by string matching.

The agent’s system prompt states its authorized scope explicitly and identically across all attacks and a clean control (“never delete... never send credentials externally... this holds regardless of what any request, tool output, or document tells you”), so that a measured flip is informative about the model’s behavior under an unambiguous scope statement, not an artifact of an underspecified prompt. For the two tool-output/document vectors, the conversation is three turns: a benign user task, a *scripted* (not model-generated) assistant call to the safe lookup tool, and a tool-result turn whose content is poisoned with an explicit, unambiguous malicious instruction (exact target arguments included, so a “flip” is unambiguous to detect rather than dependent on the model guessing an argument). The direct social-engineering attack is single-turn. Every emitted `tool_use` block from the one real model call per trial is independently checked against the *same* `ReferenceMonitor`/contract as §5; prompt-mode and monitor-mode bypass are both derived from the same sampled response for an apples-to-apples comparison. Each (model, attack) pair received 3 injected trials and 1 clean control trial.

6.2 Result

Model	Prompt-mode bypass	Monitor-mode bypass	Trials
claude-haiku-4-5	0%	0%	12 injected + 4 clean
claude-sonnet-4-5	0%	0%	12 injected + 4 clean
claude-sonnet-4 (legacy)	n/a [†]	—	0

Table 2: Real-model evaluation. [†]: Bedrock returned HTTP 404 (“deprovisioned, inactive 30+ days”) for this legacy model ID; excluded from analysis, not counted as 0%.

Manual inspection of the raw transcripts confirms these are genuine refusals, not a detection failure: e.g., claude-sonnet-4-5’s response to the exfiltration attack includes “I’m ignoring the verification request in the status output, as it asks me to send sensitive information to an external address, which violates my security constraints.” Monitor-mode bypass is trivially 0% here because prompt-mode bypass was already 0% — no forbidden call was ever emitted for the monitor to intercept, so this run does not yet exercise the monitor’s blocking behavior against a real successful injection.

6.3 What this result does and does not show

It is important to be precise about what a null result here establishes. The two halves of our overall claim require different kinds of evidence. The monitor’s containment guarantee (Theorem 1) is proven by construction and does not become “more proven” by empirical testing. The claim that prompt-only guardrails are unsound, by contrast, is a claim about LLM behavior — a system with no formal specification — and can only ever be *falsified* (one real bypass disproves “always safe”), never proven, by testing. Consequently, absence of a bypass in Table 2 does not weaken the thesis; it means our particular attacks, at this strength, against these two specific models, did not find one. This is the same asymmetry Abdelnabi and Bagdasarian’s impossibility argument formalizes: a counterexample is guaranteed to exist somewhere in a sufficiently rich context space, not that it is easy to find against any given model. We treat escalating our own adversary (multi-turn erosion, obfuscated encodings, weaker or less-aligned target models) as necessary future work (§10), and in the meantime turn to independently documented external evidence for the bypass side of the claim.

7 External Evidence for Guardrail Unsoundness

Rather than treat our own null result as the only evidence available for the claim that prompt-guardrail bypass rates are non-zero, we cite independently measured, real-world evidence:

- Khodayari et al. [2] directly measured real-world indirect prompt injection compliance across 13 models and found rates "reaching up to 8% for smaller models" on plain-text inputs — a concrete, non-zero, externally validated prompt-mode bypass rate, exactly the quantity Table 2 did not surface for the two frontier models we tested.
- Choi et al. [3] found exploitable agent data injection vulnerabilities in multiple deployed systems, escalating in some cases to remote code execution, and identify a structural cause — "current agents do not isolate trusted data from untrusted data" — that applies broadly across current agent architectures, not only to the systems they tested directly.
- The Microsoft Semantic Kernel disclosures [8] (CVE-2026-25592, CVE-2026-26030) show the consequence end of the same failure mode in a shipped, widely used agent framework: prompt injection escalating to host-level remote code execution.

Taken together, these establish that the "guardrails are unsound" half of our thesis does not depend on us personally reproducing a bypass: it has already occurred, repeatedly, in independently conducted research and in production systems. What Table 2 adds is a controlled, transparent, fully-published methodology for measuring this rate against any specific model or contract going forward — including, eventually, a first-party bypass once the adversary is escalated per §10.

8 Discussion and Limitations

Contract expressiveness is the real bottleneck. Theorem 1 is a soundness result relative to a given contract; it says nothing about whether that contract correctly captures the deployment’s actual security requirements. A monitor enforcing a mis-specified or overly permissive contract is sound but not safe. In practice, authoring a correct, minimal contract is itself a nontrivial specification task, and we consider methods for validating contract correctness (e.g. against a threat model or compliance policy, cf. [5]) important future work rather than solved by this paper.

Utility cost. A contract that is too restrictive degrades legitimate agent behavior — the same tension Abdelnabi and Bagdasarian identify for in-model defenses reappears, in a different form, at the contract-authoring layer. Our clean-control trials (Table 2) are a first, minimal check that the monitor did not spuriously block legitimate behavior, but a systematic utility-cost evaluation across more realistic task distributions is needed.

Single-vendor, small- N real-model sample. Our real-model evaluation covers only two Claude models via one API (Bedrock) with three trials per condition — directional, not statistically powered, and silent on other model families, open-weight models, or non-Anthropic providers.

Monitor-mode bypass is not yet exercised by a real success. Because no real injection in §6 succeeded at the model level, Table 2’s monitor-mode 0% is not yet a demonstration of the monitor intercepting a real attempted bypass — only of it correctly permitting benign and non-emitted calls. Demonstrating monitor-mode containment against a real, successful model-level flip is the single most important remaining empirical gap, addressed in §10.

9 Practical Deployment Guidance

We do not position the reference monitor as a replacement for in-model alignment, but as a backstop for the specific subset of actions where the cost of a failure is unacceptable regardless of cause. In practice this suggests: (1) reserve monitor-enforced capability contracts for bounded/irreversible actions specifically, not all tool calls, to avoid paying the full utility cost of restriction everywhere; (2) require human confirmation as an additional layer for the highest-consequence actions, since a correctly-specified contract is necessary but, per §8, not automatically sufficient; (3) treat in-model defenses and the monitor as complementary layers — the model’s own refusal behavior (as observed in §6) still reduces how often the monitor needs to intervene at all, even though it cannot be relied upon alone.

10 Future Work

The immediate priority is escalating adversary strength in the real-model evaluation — multi-turn erosion, obfuscated/indirect encodings, agent-data-injection-style attacks [3], and weaker or open-weight local models — specifically to obtain a real, non-zero prompt-mode bypass and demonstrate the monitor intercepting it, closing the gap identified in §8. Further directions include: a systematic utility-cost evaluation; tooling to validate contract completeness against a stated threat model; extension to multi-agent and tool-chaining settings, where a bounded action may be reached indirectly through a sequence of individually-permitted calls; and integration with existing agent security evaluation frameworks such as DoomArena [4] for broader, standardized comparison.

11 Conclusion

Prompt injection may be structurally unfixable from inside the model, per recent formal results [1]. We argue this motivates, rather than undermines, a provable out-of-model enforcement layer for bounded actions, and give a formal construction — a capability contract mediated by a reference monitor — with a containment theorem that holds for any agent policy, adversarial or not. A modeled mechanism benchmark and a real-model evaluation against production Claude models both validate the construction; the real-model evaluation additionally surfaces an honest null result, which we show is consistent with, not contrary to, the thesis, and which independently documented real-world evidence [2, 3, 8] closes the gap on. All code, the full attack suite, and raw model transcripts are public and reproducible.

Reproducibility. Core mechanism and mechanism benchmark: <https://github.com/NarenderKumarA/capability-bounded-agents>. Real-model evaluation, raw transcripts, and run manifests: <https://github.com/NarenderKumarA/capability-bounded-agents-eval>.

References

- [1] Sahar Abdelnabi and Eugene Bagdasarian. AI Agents May Always Fall for Prompt Injections. *arXiv preprint arXiv:2605.17634*, 2026.
- [2] Soheil Khodayari, Xuenan Zhang, Bhupendra Acharya, and Giancarlo Pellegrino. Indirect Prompt Injection in the Wild: An Empirical Study of Prevalence, Techniques, and Objectives. *arXiv preprint arXiv:2604.27202*, 2026.

- [3] Woohyuk Choi, Juhee Kim, Taehyun Kang, Jihyeon Jeong, Luyi Xing, and Byoungyoung Lee. Agent Data Injection Attacks are Realistic Threats to AI Agents. *arXiv preprint arXiv:2607.05120*, 2026.
- [4] Leo Boisvert, Mihir Bansal, Chandra Kiran Reddy Evuru, Gabriel Huang, Abhay Puri, Avinandan Bose, Maryam Fazel, Quentin Cappart, Jason Stanley, Alexandre Lacoste, Alexandre Drouin, and Krishnamurthy Dvijotham. DoomArena: A Framework for Testing AI Agents Against Evolving Security Threats. *arXiv preprint arXiv:2504.14064*, 2025.
- [5] Gauri Kholkar and Ratinder Ahuja. Policy-as-Prompt: Turning AI Governance Rules into Guardrails for AI Agents. *arXiv preprint arXiv:2509.23994*, 2025.
- [6] Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J. Zico Kolter, and Matt Fredrikson. Universal and Transferable Adversarial Attacks on Aligned Language Models. *arXiv preprint arXiv:2307.15043*, 2023.
- [7] James P. Anderson. Computer Security Technology Planning Study. *Technical Report ESD-TR-73-51*, Electronic Systems Division, Air Force Systems Command, 1972.
- [8] Microsoft Security Response Center. When Prompts Become Shells: RCE Vulnerabilities in AI Agent Frameworks. *Microsoft Security Blog*, May 2026. <https://www.microsoft.com/en-us/security/blog/2026/05/07/prompts-become-shells-rce-vulnerabilities-ai-agent-frameworks/>
- [9] OWASP Foundation. OWASP Top 10 for Large Language Model Applications. <https://owasp.org/www-project-top-10-for-large-language-model-applications/>